AVCI Semih
ADHAME Amine

02/06/2026

Compte Rendu - Le protocole SSH
TP n°5 - R203

Configuration de SSH

METTRE LES MACHINES SOUS DEBIAN

1 --> Vérifier l'existence des packages nécessaires openssh-server et openssh-client dans les deux machines à l'aide de la commande **dpkg -l openssh***

```
Souhait=inconnU/Installé/suppRimé/Purgé/H=à garder
| État=Non/Installé/fichier-Config/dépaqUeté/échec-conFig/H=semi-installé/W=attend-traitement-décl>
|/ Err?=(aucune)/besoin Réinstallation (État,Err: majuscule=mauvais)
||/ Nom                Version            Architecture Description
+++-==================-==================-============-=======================================>
ii  openssh-client     1:10.0p1-7+deb13u1 amd64        secure shell (SSH) client, for secure acce>
ii  openssh-server     1:10.0p1-7+deb13u1 amd64        secure shell (SSH) server, for secure acce>
ii  openssh-sftp-server 1:10.0p1-7+deb13u1 amd64       secure shell (SSH) sftp server module, for>
un  openssh-sk-helper  <aucune>           <aucune>     (aucune description n'est disponible)
~
```

Désinstaller et purger les fichiers de configuration des services SSH des deux postes :

--> On tape la commande suivante : **apt remove --purge openssh-***

```
Souhait=inconnU/Installé/suppRimé/Purgé/H=à garder
| État=Non/Installé/fichier-Config/dépaqUeté/échec-conFig/H=semi-installé/W=attend-traitement-décl>
|/ Err?=(aucune)/besoin Réinstallation (État,Err: majuscule=mauvais)
||/ Nom            Version      Architecture Description
+++-==============-============-============-=================================
un  openssh-client <aucune>     <aucune>     (aucune description n'est disponible)
un  openssh-server <aucune>     <aucune>     (aucune description n'est disponible)
lines 1-7/7 (END)
```

3. Effacer le fichier (sur les deux postes)

rm -f ~/.ssh/known_hosts

4. Réinstaller les services SSH

sudo apt update && sudo apt install openssh-*

5. Maintenant, nous allons préparer les utilisateurs SSH.

--> Sur le serveur, on créer deux utilisateurs user1 (mdp user1) et user2 (mdp user2).

Commande :
adduser user1
adduser user2
Avec le mot de passe rt pour les deux pour simplifier le TP.

AVCI Semih
ADHAME Amine
02/06/2026

6. Sur le serveur SSH :

--> redémarrer le service SSH :
sudo systemctl restart sshd

Première connexion avec mot de passe

1) Sur le client :

--> Exécuter la commande SSH pour vous connecter au serveur : ssh username@ipDuServeur

Commande :

sudo user1@172.25.0.69
mdp : rt

Que se passe t-il lors de la première connexion ?

Lors de la première connexion à l'adresse `172.25.0.69`, le client SSH affiche un avertissement indiquant l'ajout permanent de la clé publique du serveur (de type ED25519) dans le fichier `known_hosts`, avant de demander le mot de passe de l'utilisateur `user1` pour ouvrir avec succès la session sur la machine distante `iutclrtc709` sous Debian.

Vous verrez alors un message d'avertissement similaire à celui-ci :

Acceptation de la clé publique : Si vous choisissez de continuer en tapant yes, la clé publique du serveur sera ajoutée au fichier known_hosts sur votre machine locale.
Vous ne verrez pas cet avertissement lors des connexions futures à ce serveur, sauf si la clé publique du serveur change.

Pour interdire la connexion au serveur openSSH avec le login root on édite le fichier de configuration du serveur SSH /etc/ssh/sshd_config en modifiant la ligne: PermitRootLogin no
Les valeurs possibles sont : yes pour autoriser l'accès au compte root par SSH ; no pour la refuser.

BIEN TAPER LA COMMANDE POUR REDÉMARRER :

sudo systemctl restart sshd

AVCI Semih 02/06/2026
ADHAME Amine

2. Sur le client :

--> Vérifier la présence de la clé publique dans le fichier ~/.ssh/known_hosts sur le client :

nano ~/.ssh/known_hosts
On vérifie que l'adresse IP du serveur SSH avec la clé publique apparaît bien :

```
 GNU nano 8.4                                                    /root/.ssh/known_hosts
|1|syhd6qG5Fj2g3y1talyDu6sjYpM=|puoQX+PIm//P4azcm7vOAioZWYA= ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIAnbVVJKP+l94JP1KSwTVRLZl9XTkFgEWvaJwH8sQHzQ
|1|nHFFKQ2YrZomHgP5yAMcJYKNJ+s=|RtyXhx3uOdDNQHAPhDAeNkjZMPs= ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAABgQC2g33qBDPwkp4QqiAkifkB39uyQKjZdzoV/9Brws+F9PkETqr3xg1ODuTe6ccu2nBZ0YZkoCnczX3TxODpsbzhEfLOYM2/tloZI6lX3iGnVUU/4
|1|FqAAC8wjIgoHfpoDhB9Q9OmjMPI=|CzhRjwr0W1jhhyHxHkKhwHCogUA= ecdsa-sha2-nistp256 AAAAE2VjZHNhLXNoYTItbmlzdHAyNTYAAAAIbmlzdHAyNTYAAABBBJtep3wKLuVhqyUAMYwq3RRjNyNF6xsqIoadoN1gcIrbNEcJILsO4QBmSDbqJn6aY6/J48G10XPYi
|1|Kwq2apjv5mmGzM5dbzC30sb99e4=|kEBmnvWKWO57NdHr2AGTy+fud5k= ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIPOfVO7sKedcvZn5W29rI3KVrFokEIzpcG1KJWeKDVpX
|1|yfT89PpHyq5Ym+B5vh+ZpTQBq4s=|RQm1+4cHkDkVCOmOkUfqnwak4bs= ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIFYNbtj36OCdJJ+Mr3owTVEPqxlW8Aq/Qbr2lEF4iyeC
|1|oYUYKbGUr93k8728tuRCbfyp5FI=|+Oaig4XYN2++pIOZHhFDrMRhwpA= ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAABgQDJhaVpPQxVFchSHfCOTeVkFjAIpnJg1euU1wp6SylwcfsYTuhcIc63M9KhJHjAsHhRuxKv0/lEGGv7gdXZwzxMd/5/c4pmg2OyKijz1HH+EF9tE
|1|JMgFfHQudxt/w7cYbaedpHLBHC0=|Zyk3u8Z8/Z6CghowgNxqqG1nIs8= ecdsa-sha2-nistp256 AAAAE2VjZHNhLXNoYTItbmlzdHAyNTYAAABBBCpSaHJEUkzaXFmVyQHtfNhpWuZbgKE4mIL+bIiSNXAR9TRgU3e8smslwoMIEyzAUMPKEotXX4Lpt
|1|NG1JYCTHftoa4+sx/UNuc57l6LY=|Y6nylM1xp4KHKm8KBO+sSSXuESc= ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIJgldaSb96zbiCznIv5MlvhikNbjJxNL2KlUiuO3jA5P
```

En ouvrant le fichier `~/.ssh/known_hosts` avec l'éditeur `nano`, on constate que la clé publique du serveur a bien été enregistrée. L'adresse IP est automatiquement masquée (hachée) pour plus de sécurité, mais on retrouve bien le type de clé (`ssh-ed25519`) et sa signature cryptographique à la fin du fichier.

3. Effacer les clés du serveur et en régénérer d'autres en reconfigurant le serveur ssh comme suit à l'aide des commandes suivantes :

rm -v /etc/ssh/ssh_host_*

```
root@iutclrtc705:~# sudo systemctl restart sshd
root@iutclrtc705:~# rm -v /etc/ssh/ssh_host_*
'/etc/ssh/ssh_host_ecdsa_key' supprimé
'/etc/ssh/ssh_host_ecdsa_key.pub' supprimé
'/etc/ssh/ssh_host_ed25519_key' supprimé
'/etc/ssh/ssh_host_ed25519_key.pub' supprimé
'/etc/ssh/ssh_host_rsa_key' supprimé
'/etc/ssh/ssh_host_rsa_key.pub' supprimé
root@iutclrtc705:~#
```

dpkg-reconfigure openssh-server

```
root@iutclrtc705:~# dpkg-reconfigure openssh-server
Creating SSH2 RSA key; this may take some time ...
3072 SHA256:ogy6NEn3u4IzEy0MQQYrWDxbUuo7WdswDbKQtLuxSK0 root@iutclrtc705 (RSA)
Creating SSH2 ECDSA key; this may take some time ...
256 SHA256:4IqKDpBifwUr1H5a7hztKTS5UZYP1trynzy2wfmUdNI root@iutclrtc705 (ECDSA)
Creating SSH2 ED25519 key; this may take some time ...
256 SHA256:IMrmPn2mmUpdqMhG4sUlS4a5RheHi/cegyDvAL+Pbgo root@iutclrtc705 (ED25519)
ssh.socket is a disabled or a static unit not running, not starting it.
root@iutclrtc705:~#
```

systemctl restart sshd

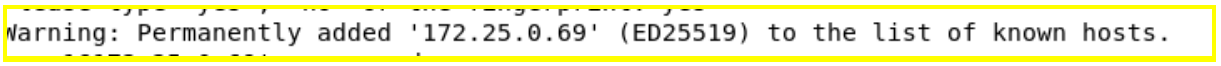
4. Sur le client, fermer la session ssh avec un **exit** et reconnectez-vous toujours avec user1.

Exit

AVCI Semih
ADHAME Amine
02/06/2026

ssh user1@172.25.0.69

Que se passe-t-il ? Quelle est la raison de cette alerte ? Lire le message d'alerte puis faire le nécessaire afin que le client puisse se reconnecter normalement au serveur.



Le client SSH refuse d'établir la connexion avec le serveur 172.25.0.69 et affiche un message d'erreur encadré par des @ : WARNING: REMOTE HOST IDENTIFICATION HAS CHANGED!. La connexion est bloquée avec un Permission denied.

L'alerte se déclenche parce que la clé publique que le serveur vient de présenter ne correspond plus à celle qui est enregistrée pour cette adresse IP dans le fichier known_hosts du client.

Pour résoudre le problème, on tape la commande suivante qui va supprimer l'ancienne clé du fichier **~/.ssh/known_hosts sur le PC Client** :

ssh-keygen

Puis on tente à nouveau le ssh :

ssh user1@172.25.0.69

Le client peut de nouveau se connecter au serveur SSH.

Connexion avec les clés

Dans cette partie, nous allons passer à une authentification plus sécurisée sans mot de passe.
Cette méthode consiste à utiliser une paire de clés (clé publique, clé privée) pour s'authentifier au serveur.

1. Toujours sur le PC client :

--> On accède au répertoire ~/.ssh :

cd ~/.ssh

--> On génère une paire de clé avec la commande #ssh-keygen sur le PC Client :

ssh-keygen

```
root@iutclrtc710:~/.ssh# ssh-keygen
Generating public/private ed25519 key pair.
Enter file in which to save the key (/root/.ssh/id_ed25519):
Enter passphrase for "/root/.ssh/id_ed25519" (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /root/.ssh/id_ed25519
Your public key has been saved in /root/.ssh/id_ed25519.pub
The key fingerprint is:
SHA256:WQmGmQQqJ9XejAjnQ/3cwXD4X6Sj1epUTjq8yZ0mFZE root@iutclrtc710
The key's randomart image is:
+--[ED25519 256]--+
|  .o.ooB+    .   |
|..o.o =oo. .E    |
|o=oo * o .o+ .   |
| ++ o = oo+ *    |
|   .    S= O .   |
|        . B o    |
|         + * .   |
|          * +    |
|           o     |
+----[SHA256]-----+
```

- un nom de fichier sera demandé (par exemple id_25519)
- choisir une passPhrase pour protéger la clé privée, que vous pouvez mémoriser facilement, il va falloir la fournir plusieurs fois.

2. Vérifier l'existence des deux clés dans le répertoire ~/.ssh et afficher leurs contenus dans le PC Serveur.

ls -l ~/.ssh

Puis on a :

```
oot@iutclrtc710:~/.ssh# ls ~/.ssh
dametoleh  adametoleh.pub  id_ed25519  id_ed25519.pub  known_hosts  known_hosts.old
oot@iutclrtc710:~/.ssh# ls -l
otal 24
rw------- 1 root root 464  1 juin  14:05 adametoleh
rw-r--r-- 1 root root  98  1 juin  14:05 adametoleh.pub
rw------- 1 root root 411  2 juin  14:33 id_ed25519
rw-r--r-- 1 root root  98  2 juin  14:33 id_ed25519.pub
rw------- 1 root root 978  2 juin  14:17 known_hosts
rw-r--r-- 1 root root 142  2 juin  14:17 known_hosts.old
```

Pour afficher le contenu de la clé privée :

cat ~/.ssh/id_ed25519

```
root@iutclrtc710:~/.ssh# cat ~/.ssh/id_ed25519
-----BEGIN OPENSSH PRIVATE KEY-----
b3BlbnNzaC1rZXktdjEAAAAABG5vbmUAAAAEbm9uZQAAAAAAAAABAAAAMwAAAAtzc2gtZW
QyNTUxOQAAACAV6rZlb5vfGzcosAJOLrX/KuZbWM1yCpOpLjuq596sXQAAAJhaHILgWhyC
4AAAAAtzc2gtZWQyNTUxOQAAACAV6rZlb5vfGzcosAJOLrX/KuZbWM1yCpOpLjuq596sXQ
AAAEAxzVfLzpflcWVyhJU6A4HMpkP8BOmCBqn+DNwlRav3LhXqtmVvm98bNyiwAk4utf8q
5ltYzXIKk6kuO6rn3qxdAAAAEHJvb3RAaXV0Y2xydGM3MTABAgMEBQ==
-----END OPENSSH PRIVATE KEY-----
```

Pour afficher le contenu de la clé publique :

AVCI Semih
ADHAME Amine

02/06/2026

cat ~/.ssh/id_rsa.pub

```
root@iutclrtc710:~/.ssh# cat ~/.ssh/id_ed25519.pub
ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIBXqtmVvm98bNyiwAk4utf8q5ltYzXIKk6kuO6rn3qxd root@iutclrtc710
root@iutclrtc710:~/.ssh#
```

3. Maintenant, nous allons copier la nouvelle clé publique du client dans le fichier .ssh/authorized_keys du répertoire personnel distant de user1

--> Saisir la commande suivante : **ssh-copy-id user1@172.25.0.69**. Cette commande :

Copie id_25519.pub vers **/home/user1/.ssh/authorized_keys** sur le serveur SSH

Crée le dossier .ssh sur le serveur SSH s'il n'existe pas

Assure les bons droits sur les fichiers/dossiers SSH

Cette commande ne peut pas s'exécuter en root.

4. Vérifier si notre clé publique a été copiée dans /home/user1/.ssh/authorized_keys

ssh user1@172.25.0.69
cat /home/user1/.ssh/authorized_keys

À l'intérieur du fichier on est censé retrouver la clé publique **id_rsa.pub**

5. Depuis la machine cliente reconnecter au serveur en SSH. Que remarquez-vous ?

ssh user1@172.25.0.69

Cette fois-ci, le serveur ne demande plus le mot de passe du compte user1.

Créer un tunnel SSH

C'est une technique qui permet de sécuriser le transfert de données entre deux machines en chiffrant le trafic réseau.
Elle permet de rediriger le trafic d'un port spécifique d'une machine locale vers une machine distante à travers une connexion SSH sécurisée.

Dans cette partie, nous allons sécuriser une connexion http en créant un tunnel ssh.

A- Connexion http sans tunnel ssh

AVCI Semih
ADHAME Amine

02/06/2026

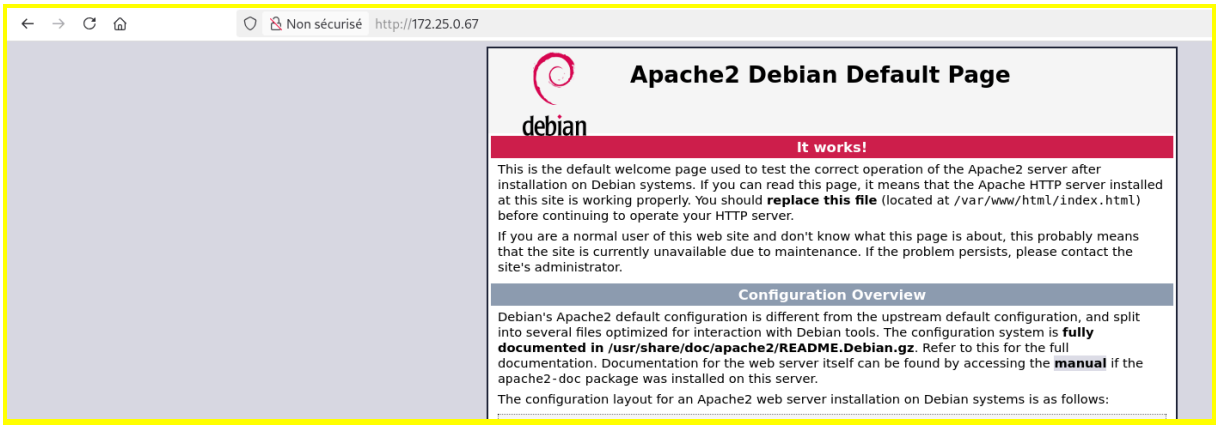
Installer et activer le service apache sur un serveur distant (une troisième machine à part du client et du serveur) :

--> apt update
--> apt install apache2

Connectez-vous au serveur apache via le navigateur de la machine cliente (avec l'adresse IP du serveur apache).

http://172.25.0.67

--> Vérifier que la page d'accueil s'affiche



Fermer la session http sur le navigateur et passer à la partie suivante.

B- Connexion http avec tunnel ssh

Mise en place du tunnel SSH pour le serveur web (et uniquement ce serveur) SUR LE CLIENT

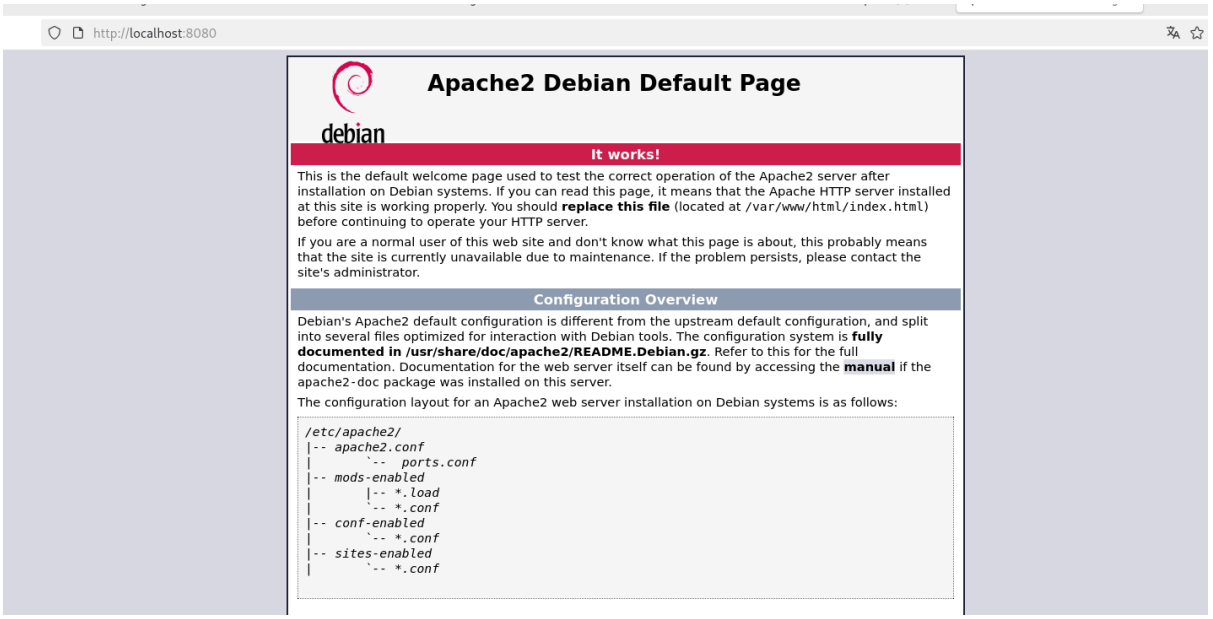
--> **ssh -L 8080:172.25.0.67:80 user1@172.25.0.69**

-L : définit la redirection locale de port
8080 : Le port locale ouvert sur la machine cliente
80 : Le port HTTP du serveur Apache

Maintenant on se reconnecte au serveur apache et vérifions que ça marche.
Pour que le tunnel s'active, il faut se connecter à l'adresse suivante : **http://localhost:8080 sur le client** Ici le tunnel est donc chiffré.

AVCI Semih
ADHAME Amine

02/06/2026

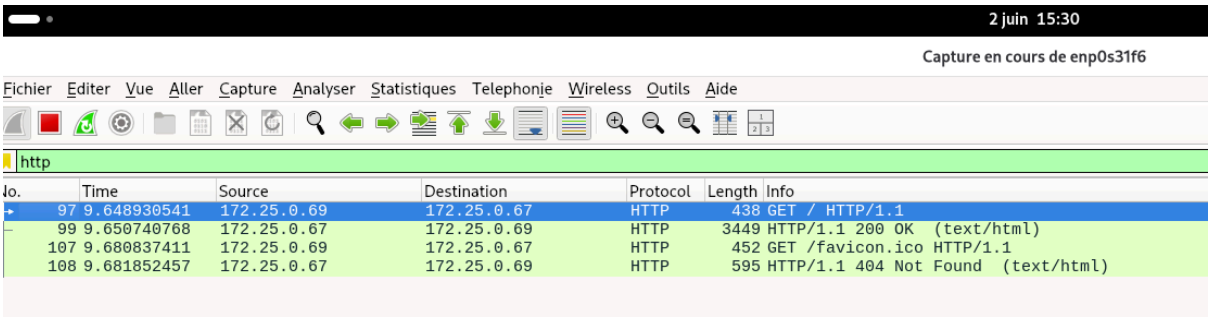


C- Mise en évidence du tunnel

On capture les trames d'une connexion http entre le client et le serveur web :
OUVRIR WIRESHARK AVANT ET FILTRER AVEC HTTP
--> URL : **http://172.25.0.67**

Taper la commande :
ssh -L 8080:172.25.0.67:80 user1@172.25.0.69
--> URL : **http://localhost:8080**

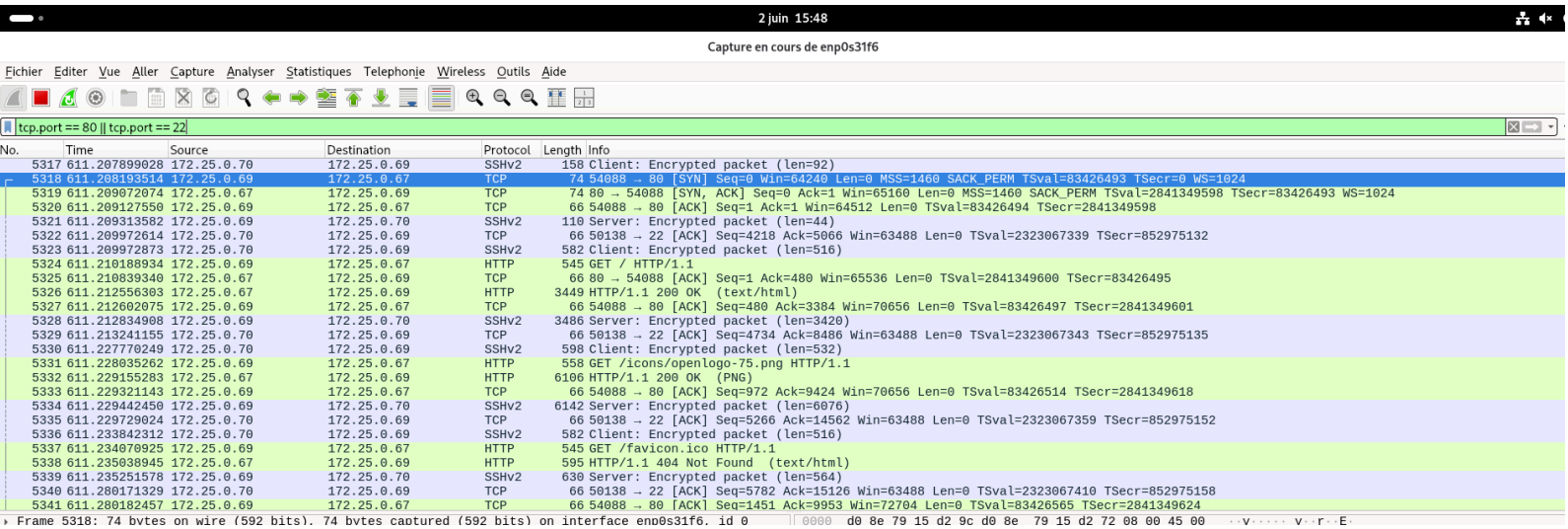
Connexion sans Tunnel :



Lors de la connexion sans tunnel, on peut voir que WIRESHARK affiche des paquets HTTP.

Cela montre que le trafic HTTP circule en clair entre le client et le serveur web.

Connexion avec tunnel :



Comparer et conclure.

Lors de la connexion avec tunnel, la page Apache s'affiche toujours correctement. Cependant, dans Wireshark, les requêtes HTTP ne sont plus visibles en clair. On observe du trafic SSH sur le port 22 entre le client et le serveur SSH. Le navigateur envoie bien une requête HTTP vers localhost:8080, mais cette requête est encapsulée dans le tunnel SSH. Elle est donc chiffrée entre le client et le serveur SSH.

On peut conclure que le tunnel SSH permet de sécuriser une connexion HTTP qui, normalement, n'est pas chiffrée.

D- Transfert de fichiers sécurisé avec ssh

Nous allons ici copier un fichier d'une machine locale vers un emplacement du serveur SSH distant via un tunnel SSH sécurisé.

La commande à taper :
scp /etc/hosts user1@172.25.0.69:/home/user1

scp = secure copy

/etc/hosts = fichier local à copier

user1@172.25.0.69 = utilisateur et IP du serveur SSH

/home/user1 = répertoire de destination sur le Serveur SSH distant

Pour vérifier le bon fonctionnement, on peut écrire sur le terminal :

AVCI Semih
ADHAME Amine

02/06/2026

ls -l /home/user1

```
user1@iutclrtc709:~$ ls -l /home/user1
total 4
-rwxr-xr-x 1 user1 user1   0  6 mai   15:53 fich1.txt
-rw-r--r-- 1 user1 user1 217  2 juin  15:50 hosts
user1@iutclrtc709:~$
```

```
root@iutclrtc710:~/.ssh# scp /etc/hosts user1@172.25.0.69:/home/user1
hosts                                                                                                100%  217   196.9KB/s   00:00
root@iutclrtc710:~/.ssh#
```